

Predictra Threat Map Frontend

Fullstack Synapse & Interactive Visual Guide

This comprehensive guide details the frontend architecture, performance throttles, 3D graphics rendering, and fullstack connection mechanisms of the Predictra Cyber Threat Map. It contains an intensive bank of 25 active recall questions to ensure complete structural comprehension and technical memory consolidation.

React + TypeScript + Three.js + Zustand + Express + MongoDB

June 2026

Contents

1	The Frontend Interface (The Sensory Display)	2
1.1	The Core Tech Stack	2
1.2	The Interface Views	2
2	State Management & Event Ingestion	4
2.1	State Variables & Data Models	4
2.2	The SSE Connection Pipeline	4
2.3	Exponential Reconnection Backoff	5
3	Visual Performance Optimizations	6
3.1	1. Dynamic Adaptive Sampling (FPS Guard)	6
3.2	2. Staged Batch Throttling	6
3.3	3. Hard Visual Limits	7
3.4	4. Geometry Calculations: The Arc Path	7
4	The STIX Intelligence Engine	8
4.1	STIX Bundle Parsing	8
4.2	MITRE ATT&CK Kill Chain Mapping	8
4.3	Indicator of Compromise (IOC) Classification	9
5	The Fullstack Synapse	10
5.1	The Data Key Mapping Matrix	10
5.2	The Excel Export Pipeline	11
6	Active Recall & Memory Consolidation	12
6.1	25 Core Study Questions	12
6.2	Answer Key & Explanations	14

Chapter 1

The Frontend Interface (The Sensory Display)

While the backend processes threat telemetry, the frontend renders it. The frontend is built as a single-page application (SPA) using React, TypeScript, and Vite. Its core mission is to render cyber events in a way that provides instant situational awareness.

1.1 The Core Tech Stack

The visualizer runs on four main pillars:

1. **React (v19.2) & TypeScript:** Core engine and type-safe component hierarchy.
2. **React Three Fiber (R3F) & Drei (v10.7):** Declaring Three.js WebGL scenes in React syntax.
3. **Zustand (v5.0):** The lightweight, fast state manager that processes real-time event updates and runs the UI thread.
4. **D3 (d3-force, d3-shape):** Math utilities for complex geometry computations.

1.2 The Interface Views

The application supports six interface views, switched dynamically via the sidebar:

- **Map View ('map'):** Renders the full 3D Earth globe with attack arcs flying from origin to target. Includes HUD overlays with live threat gauges, system statuses, sparklines, and active alerts.
- **Dashboard View ('dashboard'):** An alternative flat panel showing structured summaries: attack types distribution, top vectors, primary target countries, major origin locations, and attack corridors. Uses a stylized SVG world map background with scanline overlays.

- **History View ('history'):** A tabular log browser containing a searchable list of database records. Supports regex search querying against target IPs, source IPs, CVEs, or tags.
- **Country Focus View ('country'):** Displays a dedicated hologram panel focusing on a single nation's threat vectors, active origins, and target corridors.
- **Analytics View ('analytics'):** Aggregates historic databases into heatmaps, timeline charts, and sector matrix tables.
- **STIX Dashboard ('stix'):** Processes and displays standard STIX JSON threat bundles (e.g., CISA reports, Ivanti exploits), mapping indicators to the MITRE ATT&CK Kill Chain.

Memory Anchor: The Display Metaphor

Think of the UI as a ****Tactical Hologram Table****. Instead of simple data lists, it is a cockpit: Map view is the 3D map, Dashboard is the radar console, History is the archive logs, and STIX is the adversary intelligence folder.

Active Recall Challenge: UI Views Inventory

1. Name all 6 views available in the Predictra interface.
2. What is the fundamental difference in rendering layout between the 'map' view and the 'dashboard' view?

Chapter 2

State Management & Event Ingestion

At the core of the frontend is `src/stream/useStreamStore.ts`. This is a Zustand store that handles the Server-Sent Events (SSE) feed, maintains threat queues, tracks system metrics, and coordinates UI transitions.

2.1 State Variables & Data Models

The store tracks:

- **status**: SSE connection state (`'disconnected'`, `'reconnecting'`, `'live'`).
- **arcs[] & markers[]**: Active graphical objects rendered on the 3D canvas.
- **eventBuffer**: A custom circular ring-buffer (`RingBuffer`) that stores up to 10,000 events in memory for fast history retrieval.
- **typeDistribution, originDistribution, targetDistribution**: Cached counters of threat characteristics, avoiding database queries on every click.
- **trendData[]**: An array of 60 buckets tracking threats in 10-second intervals to render trend sparklines.

2.2 The SSE Connection Pipeline

When the store initializes via `initStream()`, it instantiates an `EventSource` connection to the backend endpoint (defaults to `/api/feed`):

```
1 eventSource = new EventSource(apiUrl);
2 eventSource.addEventListener('attacks', (e: MessageEvent) => {
3   const batch = JSON.parse(e.data);
4   if (!Array.isArray(batch)) return;
5   const validEvents = [];
6   for (const data of batch) {
7     if (!data.a_t || data.s_la === undefined || data.d_la === undefined)
        continue;
```

```

8     validEvents.push({
9         id: fastId(),
10        a_c: data.a_c || 1,
11        a_n: String(data.a_n).slice(0, 200),
12        a_t: data.a_t,
13        s_co: data.s_co, s_la: Number(data.s_la), s_lo: Number(data.s_lo),
14        d_co: data.d_co, d_la: Number(data.d_la), d_lo: Number(data.d_lo),
15        s_ip: data.s_ip, d_ip: data.d_ip,
16        source_api: data.source_api,
17        ts: new Date().toISOString(),
18        meta: data.meta || {}
19    });
20 }
21 if (validEvents.length > 0) get().addEvents(validEvents);
22 });

```

2.3 Exponential Reconnection Backoff

If the backend crashes or restart occurs, the store protects the network by avoiding immediate reconnections. If connection fails, it triggers a retry with exponential backoff:

```

1 // Backoff with Jitter
2 const jitter = Math.random() * 1000;
3 setTimeout(connect, reconnectDelay + jitter);
4 reconnectDelay = Math.min(reconnectDelay * 2, 30000);

```

The delay starts at 1,000ms, doubles on every failure (up to a 30-second cap), and adds a randomized jitter offset. This prevents "connection swarms" when the server recovers.

Active Recall Challenge: Connection & Event Loop

1. What is the role of RingBuffer in useStreamStore? Why use it instead of a standard array?
2. Explain what exponential backoff with jitter is and why it is used in the frontend stream connector.

Chapter 3

Visual Performance Optimizations

Rendering hundreds of flying lines and flashing markers in WebGL in real-time is computationally demanding. Without optimization, frame rates would drop below 15 FPS, causing visual lag. Predictra implements three performance optimizations:

3.1 1. Dynamic Adaptive Sampling (FPS Guard)

The store monitors the client's actual frame rate using a custom telemetry class `perfTelemetry`. When adding new threat events to the map, it dynamically drops visual elements if performance degrades:

```
1 const fps = perfTelemetry.stats.fps;
2 let visualEvents = events;
3 if (fps < 30) {
4   visualEvents = events.filter((_, i) => i % 3 === 0); // Keep only 33%
      of arcs
5 } else if (fps < 45) {
6   visualEvents = events.filter((_, i) => i % 2 === 0); // Keep only 50%
      of arcs
7 }
```

If the machine lags ($\text{FPS} < 30$), the store filters out two-thirds of the graphical arcs. This keeps the animation smooth while retaining accurate statistical counting (the counters in Zustand still increment by the true, unfiltered amount).

3.2 2. Staged Batch Throttling

To prevent React from re-rendering the UI dozens of times per second, the store separates high-frequency event updates from state commits:

- **Recent Event Updates:** Throttled to a maximum of one commit every 500ms.
- **Analytics Matrix Re-calculations:** Accumulated in temporary global variables (e.g. `pendingType`, `pendingOrigin`) and flushed to React state only once per second.

3.3 3. Hard Visual Limits

Graphical objects are hard-capped in size to prevent GPU memory bloat:

- `MAX_ARCS` = 150 (maximum simultaneous flying curves).
- `MAX_MARKERS` = 300 (maximum simultaneous flashing impact markers).

Once these thresholds are reached, oldest visual elements are deleted immediately.

3.4 4. Geometry Calculations: The Arc Path

The 3D arc path between coordinates is generated in `src/utils/geo.ts` using spherical interpolation (Slerp) combined with a parabolic elevation equation:

$$\text{Elevation}(t) = \sin(t \cdot \pi) \cdot \text{maxArcHeight} \quad (3.1)$$

As progress (t) goes from 0 to 1, the elevation follows a sine curve, peaking at $t = 0.5$. This creates the curved trajectory above the Earth's surface.

Active Recall Challenge: Optimization Analysis

1. What happens to visual arc rendering when the client FPS drops to 25? What happens to the total threat counters?
2. Why are analytics calculations throttled to a 1-second interval?
3. Write down the equation used to calculate arc elevation at a given progress fraction t .

Chapter 4

The STIX Intelligence Engine

Predictra is not just a live threat visualizer; it is an intelligence platform. The STIX Dashboard (`src/ui/StixDashboard.tsx`) parses standard cybersecurity documents and correlates threat indicators.

4.1 STIX Bundle Parsing

STIX (Structured Threat Information Expression) represents threat details using JSON objects linked by relationships. The parser maps:

- **reports:** The core dossiers containing metadata.
- **attack-patterns:** Specific tactics, techniques, and procedures (TTPs) used by adversaries (e.g. SQL Injection, Web Shell upload).
- **indicators:** Artifacts that indicate compromise (e.g. registry keys, filenames, domain names).

4.2 MITRE ATT&CK Kill Chain Mapping

The dashboard parses the name and descriptions of techniques to categorize them into standard kill chain phases (Reconnaissance, Delivery, Execution, Persistence, Lateral Movement, Command & Control):

```
1 const KILL_CHAIN_PHASES = [  
2   { id: 'recon', label: 'Reconnaissance', keywords: ['reconnaissance', '  
     scanning'], color: '#8B5CF6' },  
3   { id: 'initial', label: 'Initial Access', keywords: ['initial access',  
     'exploit public-facing'], color: '#EF4444' },  
4   // ...  
5 ];
```

If keywords are matched, the technique is mapped to the corresponding phase in the visual matrix.

4.3 Indicator of Compromise (IOC) Classification

Indicators are written in STIX pattern syntax (e.g. "[file:hashes.'SHA-256' = 'a1b2...']"). The frontend uses a regex classifier to extract clean indicators:

```

1 function classifyIndicator(obj: StixObject) {
2   const pattern = obj.pattern || '';
3   if (pattern.includes('ipv4-addr')) {
4     const match = pattern.match(/(\d+\.\d+\.\d+\.\d+)/);
5     return { type: 'IPv4', value: match ? match[1] : pattern, icon: 'IP'
6           };
7   }
8   if (pattern.includes('file:hashes')) {
9     const sha256 = pattern.match(/SHA-256[^\s]*='([~']+)'/i);
10    return { type: 'SHA-256', value: sha256 ? sha256[1] : pattern, icon:
11          'File' };
12  }
13  // Mutex, Domain, URL classification...
14 }

```

Active Recall Challenge: STIX Parsing

1. What does STIX stand for? What is the main objective of the STIX view?
2. How does the frontend classify and display hashes vs IP addresses in the IOC tab?
3. Name three phases of the MITRE ATT&CK Kill Chain defined in the store.

Chapter 5

The Fullstack Synapse

Now let's examine the connection between the frontend and the backend. They operate in a decoupled client-server architecture:

Frontend React App	Transmission	Backend Express App
<code>initStream()</code> listeners	← SSE Stream ←	<code>/api/feed</code> (300ms flushes)
Zustand stats state	← HTTP Poll ←	<code>/api/stats</code> (MongoDB aggregation)
ExcelJS / SaveAs exports	← User Request ←	Document downloads
History page regexp search	→ HTTP GET →	<code>/api/history?q=...</code> search
Toggle database controls	→ HTTP POST →	<code>/api/db/toggle</code> state change

Figure 5.1: Fullstack Communication Matrix

5.1 The Data Key Mapping Matrix

To maintain the optimized database footprint, the backend saves data using abbreviated keys. The frontend maps these database attributes directly to human-readable variables:

MongoDB Document Key	Synapse Transmission	React Rendering Variable
<code>a_c</code>	→	Attack occurrences count
<code>a_n</code>	→	Threat signature / name
<code>a_t</code>	→	Threat classification (<code>exploit</code> , <code>malware</code> , <code>phishing</code>)
<code>s_co</code>	→	Source Country flag and label
<code>s_la</code> / <code>s_lo</code>	→	Globe 3D coordinates (Slerp start point)
<code>d_co</code>	→	Target Country flag and label
<code>d_la</code> / <code>d_lo</code>	→	Globe 3D coordinates (Slerp end point)
<code>meta.organization</code>	→	Vector organization owner (RDAP resolve)
<code>source_api</code>	→	Threat intelligence provider label

5.2 The Excel Export Pipeline

The frontend allows administrators to export logs to Excel via ‘exceljs’ in `DashboardPage.tsx`.

1. The app extracts current threat events from the store’s circular buffer.
2. It instantiates a workbook structure in memory.
3. It styles header fields with dark blue fills and sets standard column widths.
4. It iterates over threat rows. It applies conditional formatting, highlighting exploit rows in red, malware in gold, and phishing in violet.
5. It compiles the worksheet into a binary blob and uses ‘file-saver’ to download the file directly to the client’s machine.

Active Recall Challenge: Synapse Mechanics

1. Complete the mapping: What does `s_1a` represent on the 3D globe scene?
2. How does the backend quota monitor (500MB cap) affect what is displayed in the frontend?
3. Explain the steps the client takes to export threat data into a formatted Excel sheet.

Chapter 6

Active Recall & Memory Consolidation

Answer these questions to test your knowledge of the Predictra frontend and fullstack synapse. Write down your answers before checking the key on the next page.

6.1 25 Core Study Questions

1. **Stack:** Which state management library handles the frontend stream data?
2. **Vite:** What does Vite do in the context of the frontend app?
3. **Engine:** What framework maps WebGL 3D canvas coordinates to React components?
4. **Store:** Which file manages the main SSE connection and state in the app?
5. **Buffer:** What is the maximum size capacity of the store's circular 'eventBuffer'?
6. **Visuals:** What is the maximum number of simultaneous arcs allowed on the globe?
7. **Visuals:** What is the maximum number of simultaneous markers allowed on the globe?
8. **API:** What is the default path of the Server-Sent Events stream on the backend?
9. **Backoff:** What delay is used for the first reconnection attempt after a failure?
10. **Backoff:** What is the maximum retry delay cap of the exponential backoff loop?
11. **Backoff:** Why is randomized "jitter" added to the reconnection timeout?
12. **Graphics:** Below what FPS threshold does the store activate the 33% sampling rate?
13. **Graphics:** What is the sampling rate when frame rate is between 30 and 45 FPS?
14. **Graphics:** Why does the app drop visual arcs while maintaining the main counters?
15. **Throttling:** How often are recent events updated in the UI store?

16. **Throttling:** How often are the analytics maps flushed to the React state?
17. **Math:** What mathematical function is used to calculate elevation over the globe?
18. **STIX:** Which two STIX threat bundles are imported by default in the Stix view?
19. **STIX:** What regex check is used to extract file hashes from STIX patterns?
20. **STIX:** What does the STIX parser do with the ‘relationship’ data type?
21. **Synapse:** What port does the backend SSE feed serve by default?
22. **Synapse:** What database property translates to the threat class label in React?
23. **Synapse:** Which API endpoint is queried when searching history logs?
24. **Synapse:** Why are MongoDB inserts bulk-saved in batches of 25 instead of saved individually?
25. **Export:** Which npm libraries compile and download styled spreadsheet reports in the client?

6.2 Answer Key & Explanations

1. **Zustand** manages the stream data state.
2. **Vite** compiles, bundles, and serves the frontend React application.
3. **React Three Fiber (R3F)** manages WebGL scenes in React.
4. **src/stream/useStreamStore.ts** manages connections and state.
5. **10,000 events** is the maximum capacity of the circular buffer.
6. **150 arcs** is the maximum simultaneous arc limit.
7. **300 markers** is the maximum simultaneous marker limit.
8. **/api/feed** is the default path of the SSE stream.
9. **1,000ms** is the initial retry delay.
10. **30,000ms** (30 seconds) is the maximum retry delay.
11. To prevent all client devices from reconnecting at the same millisecond and overloading the server.
12. Below **30 FPS**, the app drops to a 33% sampling rate.
13. **50%** of visual arcs are rendered when FPS is between 30 and 45.
14. To protect visual performance while maintaining accurate statistics.
15. Every **500ms**.
16. Every **1,000ms** (1 second).
17. A ****Sine function**** (`Math.sin(t * Math.PI)`) elevates arcs above the globe.
18. **cisa.json** and **ivanti.json** are imported.
19. Looks for the **file:hashes** string and extracts associated hash values.
20. Maps links between threat reports, attack patterns, and indicator nodes.
21. Port **3001** is the default backend port.
22. The **a_t** key maps to threat types.
23. **/api/history?q=...** queries historical database entries.
24. To avoid database locking and network latency overhead.
25. **exceljs** and **file-saver** compile and download files.